

Hamiltonian Paths from Overlapping Consecutive Sets

Ready for expert review
2026-09-22

Abstract

We give deterministic polynomial-time construction and recovery maps from Hamiltonian Path to Consecutive Sets. Every vertex receives a private symbol and every graph edge supplies one symbol shared by its endpoints' sets. A short target string must save $n - 1$ positions by overlapping set-occurrence windows. Their interval graph is a forest, while singleton overlaps at interval endpoints give maximum degree two; it is therefore a Hamiltonian path. The decoder works for every valid target string, including strings with repeated symbols and sets with multiple occurrences. Independent verification covers every simple graph through four vertices, 525 recoveries, and 456 repeated-symbol outputs.

1. Introduction

Consecutive Sets asks for a bounded-length string in which every supplied set appears as a contiguous block, in any order and without repetition inside that block. Symbols may repeat elsewhere. The problem is the bounded duplication version of consecutive information retrieval: records may be copied so that every query's records occupy one interval.

Kou's Theorem 5 gives the exact construction used here for the corresponding minimum-duplication problem, by reduction from Hamiltonian Path [1, printed p. 74]. Garey and Johnson catalogue the bounded decision form as SR18 [2]. The motivating issue proposes closed vertex neighborhoods, but non-path graph edges can make such neighborhoods nonconsecutive; its own example exhibits that failure [3].

We reconstruct the reduction through set overlaps and state it for the fixed search contract. The proof treats a target output as an arbitrary string, rather than a permutation of the alphabet, and recovers a path from any choices of the required occurrence windows.

Main theorem. There are deterministic polynomial-time maps F and G from Hamiltonian Path to Consecutive Sets such that, for every legal source input x and every valid target output y of $F(x)$, $G(x, y)$ is a valid source output. This includes arbitrary valid strings and NO-SOLUTION.

2. Definitions

A source instance is a finite simple undirected graph $G = (V, E)$. A positive output orders every vertex exactly once and uses a graph edge between each consecutive pair. The empty graph accepts the empty path. NO-SOLUTION is valid exactly when no Hamiltonian path exists.

A target instance has a finite alphabet, an explicit family of its subsets and a nonnegative integer K . A positive output is a string of length at most K over the alphabet. For every supplied set S , some window of length $|S|$ must contain every symbol of S exactly once. Symbols may occur elsewhere, family members may repeat, and the empty set has an empty occurrence.

3. Construction and recovery

For every vertex v , create a private symbol p_v . For every edge e , create one edge symbol x_e . Supply the set

$$S_v = \{p_v\} \cup \{x_e : e \text{ is incident with } v\}. \quad (1)$$

The alphabet contains the $n + m$ symbols just created. For the empty graph set $K = 0$. Otherwise set

$$K = \sum_{v \in V} |S_v| - (n - 1) = 2m + 1. \quad (2)$$

Two vertex sets share one symbol exactly when their vertices are adjacent. No symbol occurs in three supplied sets.

Given a positive target string, the decoder picks the first occurrence window of every S_v , joins two vertices when their chosen intervals overlap, and traverses the resulting graph from an endpoint. A valid target NO-SOLUTION maps directly to source NO-SOLUTION.

4. Correctness

4.1. Forward witness

Lemma 1. Every Hamiltonian path yields a target string of length K .

Proof. Let $v_1, \dots, v_n$ be a Hamiltonian path. In each internal block S_{v_i} put the predecessor edge symbol first and successor edge symbol last; put the other symbols between them. Use the successor symbol last in the first block and the predecessor symbol first in the last. Overlay consecutive blocks on their common path-edge symbol. This saves one position for each of the $n - 1$ path edges, giving Equation 2. Every supplied set remains an exact contiguous block. The empty and singleton cases are immediate. $\square$

4.2. Occurrence intervals

Fix any valid target string w and choose any occurrence interval I_v for each set. Let H be their intersection graph and let U count string positions covered by at least one chosen interval.

Lemma 2. The graph H is a forest, and every pair of its intervals intersects in exactly one position.

Proof. At most two chosen intervals cover one position, since its character belongs to all corresponding sets and no construction symbol belongs to three. Two intervals meet in at most one position: their sets share at most one symbol, which cannot repeat inside either window.

A triangle would give three pairwise-intersecting intervals and hence, by the Helly property, one position common to all three. If a shortest cycle had length at least four, choose on it an interval with minimum right endpoint. Both cycle neighbors contain that endpoint and create a chord. Both cases contradict the preceding coverage facts, so H is a forest. $\square$

4.3. The length bound forces a path

Lemma 3. The graph H is a spanning path whose consecutive vertices are adjacent in G .

Proof. Let c_j count chosen intervals at position j . Because c_j is one or two on the covered positions, and double-covered positions correspond bijectively to edges of H ,

$$|E(H)| = \sum_v |S_v| - U \geq \sum_v |S_v| - |w| \geq n - 1. \quad (3)$$

Lemma 2 makes H a forest, so it has at most $n - 1$ edges. Equality holds and H is connected.

Every interval participating in an overlap has length at least two: its vertex has the shared input edge in addition to its private symbol. Two intervals of length at least two with a singleton intersection meet at one endpoint of each. Each interval has two endpoints, and no endpoint belongs to three intervals, so H has maximum degree two. A connected tree of maximum degree two is a spanning path. Intersecting vertex sets share their unique edge symbol, so every step of that path is an input edge. $\square$

Theorem 4. The construction and decoder satisfy the main theorem.

Proof. Lemma 1 proves forward feasibility. Lemma 3 applies to any choice of occurrence windows and validates every positive recovery, including when symbols repeat elsewhere. The two directions imply that a valid target NO-SOLUTION is also valid for the source. $\square$

5. Complexity, provenance and limits

The target has $n + m$ symbols, n sets and total family size $n + 2m$; for $n > 0$, $K = 2m + 1$. Construction is linear in the explicit graph size. Direct validation scans a polynomial number of windows, and recovery compares $O(n^2)$ interval pairs before traversing the path.

Kou’s Theorem 5 uses records $R = E \cup V$, gives each vertex query its private vertex record and incident-edge records, and sets the bound to $1 - n + \sum_v |S_v|$ [1, printed p. 74]. Thus the gadget and bound are Kou’s. The present treatment supplies a complete interval-forest converse for the fixed bounded-string semantics and an explicit decoder for every valid output. The result is a reconstruction of known work, not a new complexity classification.

6. Verification and reproducibility

The prepared source and target oracles were written before the candidate. Their self-test cross-checks Z3 with brute force on ten source cases and seven target fixtures. The prepared closed loop passed 34 recoveries.

A separate subprocess verifier independently models target strings and checks every simple graph through four vertices. It passed 76 instances and 525 recoveries; 456 positive outputs repeated at least one symbol. Finite checks support the implementation. A fresh registered reviewer independently checked all 34 valid strings generated on graphs through three vertices, including 24 with repeated symbols, plus the four-vertex star and malformed CLI inputs. The review advanced after the manuscript explicitly attributed Kou’s exact gadget and bound. Theorem 4 supplies the general guarantee, including strings with multiple possible occurrence windows.

From the repository root, run:

```
uv run --locked python campaigns/hamiltonian-path-consecutive-sets/work/check.py --self-test
uv run --locked python campaigns/hamiltonian-path-consecutive-sets/work/check.py \
--candidate campaigns/hamiltonian-path-consecutive-sets/work/algorithm.py
uv run --locked python campaigns/hamiltonian-path-consecutive-sets/work/verify.py
uv run --locked python campaigns/hamiltonian-path-consecutive-sets/reviews/initial/targeted_check.py
```

The locked environment uses CPython 3.14.2 and z3-solver 5.1.0.0 reporting Z3 5.1.0. This manuscript was compiled with Typst 0.15.1.

References

- [1] L. T. Kou. “Polynomial Complete Consecutive Information Retrieval Problems.” *SIAM Journal on Computing* 6(1):67–75, 1977. Theorem 5, printed p. 74. DOI: 10.1137/0206004.
- [2] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979. Entry SR18.
- [3] CodingThrust. “Hamiltonian Path to Consecutive Sets.” Problem-Reductions issue 436, opened 2026. github.com/CodingThrust/problem-reductions/issues/436.